中国海洋大学

计算机网络 课程实验报告

实验题目： 通过 Wiresahrk 抓包工具深度剖析 ICMP 协议

实验概述

【实验目的及要求】

（1）Ping 是如何利用 ICMP 消息工作的

（2）traceroute 使用和利用 ICMP 消息工作的

（3）ICMP 消息的具体格式和内容

【实验环境】实验用的软硬件环境（配置）

1、硬件环境：

- 设备类型：个人计算机（PC）
- 操作系统：Microsoft Windows 10 家庭中文版
- 处理器：Intel(R) Core(TM) i7-8565U CPU @ 1.80GHz，1992 Mhz，4 个内核，8 个逻辑处理器
- 存储：477 GB SSD WDC PC SN720 SDAPNTW-512G-1127
- 显卡：NVIDIA GeForce MX250 (2 GB), Intel(R) UHD Graphics 620 (128 MB)
- 网络适配器：Intel(R) Wireless-AC 9560 160MHz

2、软件环境

- 网络抓包工具：Wireshark 4.6.0 (v4.6.0-0-gcdfb6721e77c).（支持 TCP/UDP/ICMP 协议解析，捕获以太网 / Wi-Fi 数据包）

实验内容

【实验方案设计】

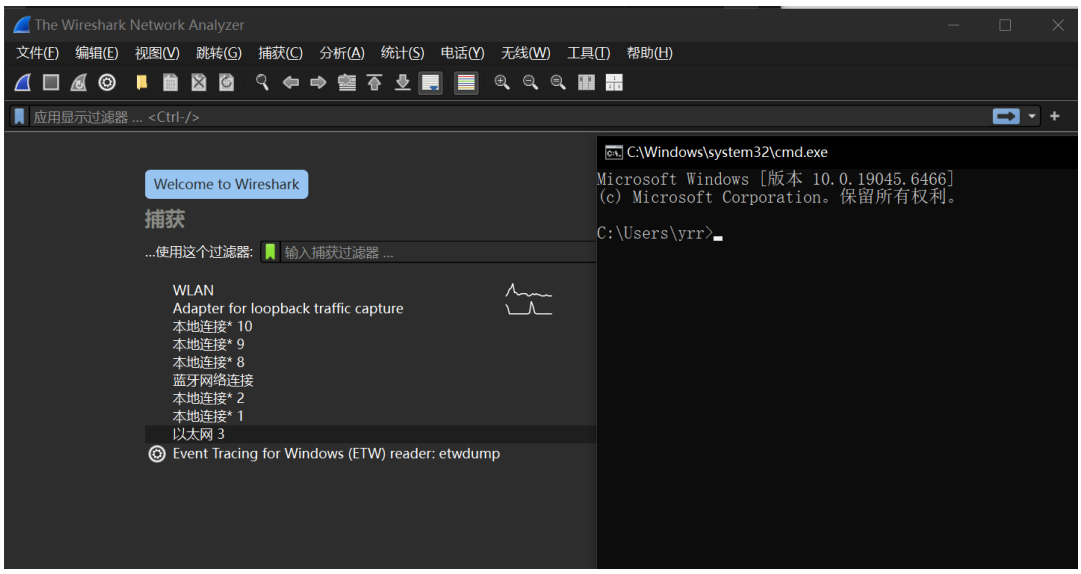
实验任务 1：ICMP AND Ping

1.1 了解 ping

Ping 程序是网络管理员等人员用来检测主机是否在线的简单工具。源主机的 Ping 程序会向目标 IP 地址发送数据包，若目标主机在线，目标主机的 Ping 程序就会回传数据包。这两个 Ping 数据包都是 ICMP 数据包。

1.2 使用 cmd ping 远程主机，并通过 wireshark 获取 ICMP 数据包

1.2.1 打开 cmd 和 wireshark



1.2.2 捕获数据包

启动 wireshark，并通过 ping 程序，ping 一个跨洋的远程主机，用于产生跨洋流量以便观察 RTT。ping 程序终止时，停止捕获。

ping 结果显示如下，一共向 mit 服务器发送了 10 个 ping 包，并收到了 10 个响应：

```
C:\Users\yrr>ping -n 10 www.mit.edu

正在 Ping e9566.dscb.akamaiedge.net [23.34.43.128] 具有 32 字节的数据:
来自 23.34.43.128 的回复: 字节=32 时间=235ms TTL=65
来自 23.34.43.128 的回复: 字节=32 时间=235ms TTL=65
来自 23.34.43.128 的回复: 字节=32 时间=230ms TTL=65
来自 23.34.43.128 的回复: 字节=32 时间=228ms TTL=65
来自 23.34.43.128 的回复: 字节=32 时间=228ms TTL=65
来自 23.34.43.128 的回复: 字节=32 时间=239ms TTL=65
来自 23.34.43.128 的回复: 字节=32 时间=221ms TTL=65
来自 23.34.43.128 的回复: 字节=32 时间=230ms TTL=65
来自 23.34.43.128 的回复: 字节=32 时间=226ms TTL=65
来自 23.34.43.128 的回复: 字节=32 时间=224ms TTL=65

23.34.43.128 的 Ping 统计信息:
    数据包: 已发送 = 10，已接收 = 10，丢失 = 0 (0% 丢失)，
往返行程的估计时间(以毫秒为单位):
    最短 = 221ms，最长 = 239ms，平均 = 229ms
```

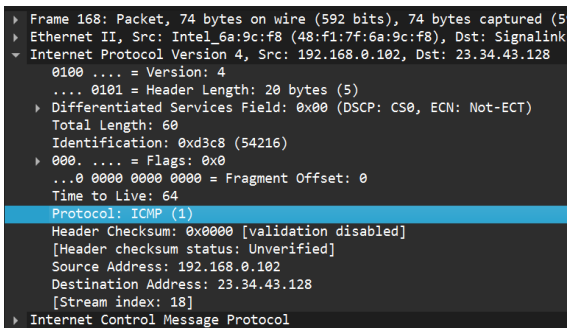
丢失率为 0%，平均 RTT 为 229ms。

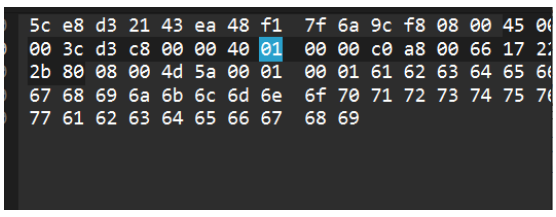
在过滤器显示窗口中输入“icmp”后的 Wireshark 输出截图如下：

168 42.924855	192.168.0.102	23.34.43.128	ICMP	74 Echo (ping) request	id=0x0001, seq=1/256, ttl=64 (reply in 169)
169 43.160023	23.34.43.128	192.168.0.102	ICMP	74 Echo (ping) reply	id=0x0001, seq=1/256, ttl=65 (request in 168)
171 43.937823	192.168.0.102	23.34.43.128	ICMP	74 Echo (ping) request	id=0x0001, seq=2/512, ttl=64 (reply in 173)
173 44.173276	23.34.43.128	192.168.0.102	ICMP	74 Echo (ping) reply	id=0x0001, seq=2/512, ttl=65 (request in 171)
174 44.959074	192.168.0.102	23.34.43.128	ICMP	74 Echo (ping) request	id=0x0001, seq=3/768, ttl=64 (reply in 178)
178 45.188921	23.34.43.128	192.168.0.102	ICMP	74 Echo (ping) reply	id=0x0001, seq=3/768, ttl=65 (request in 174)
180 45.967606	192.168.0.102	23.34.43.128	ICMP	74 Echo (ping) request	id=0x0001, seq=4/1024, ttl=64 (reply in 181)
181 46.195954	23.34.43.128	192.168.0.102	ICMP	74 Echo (ping) reply	id=0x0001, seq=4/1024, ttl=65 (request in 180)
188 46.991822	192.168.0.102	23.34.43.128	ICMP	74 Echo (ping) request	id=0x0001, seq=5/1280, ttl=64 (reply in 189)
189 47.219745	23.34.43.128	192.168.0.102	ICMP	74 Echo (ping) reply	id=0x0001, seq=5/1280, ttl=65 (request in 188)
190 48.001561	192.168.0.102	23.34.43.128	ICMP	74 Echo (ping) request	id=0x0001, seq=6/1536, ttl=64 (reply in 191)
191 48.240953	23.34.43.128	192.168.0.102	ICMP	74 Echo (ping) reply	id=0x0001, seq=6/1536, ttl=65 (request in 190)
193 49.041141	192.168.0.102	23.34.43.128	ICMP	74 Echo (ping) request	id=0x0001, seq=7/1792, ttl=64 (reply in 195)
195 49.262688	23.34.43.128	192.168.0.102	ICMP	74 Echo (ping) reply	id=0x0001, seq=7/1792, ttl=65 (request in 193)
198 50.057366	192.168.0.102	23.34.43.128	ICMP	74 Echo (ping) request	id=0x0001, seq=8/2048, ttl=64 (reply in 199)
199 50.287633	23.34.43.128	192.168.0.102	ICMP	74 Echo (ping) reply	id=0x0001, seq=8/2048, ttl=65 (request in 198)
200 51.076382	192.168.0.102	23.34.43.128	ICMP	74 Echo (ping) request	id=0x0001, seq=9/2304, ttl=64 (reply in 201)
201 51.302655	23.34.43.128	192.168.0.102	ICMP	74 Echo (ping) reply	id=0x0001, seq=9/2304, ttl=65 (request in 200)
202 52.080542	192.168.0.102	23.34.43.128	ICMP	74 Echo (ping) request	id=0x0001, seq=10/2560, ttl=64 (reply in 203)
203 52.304726	23.34.43.128	192.168.0.102	ICMP	74 Echo (ping) reply	id=0x0001, seq=10/2560, ttl=65 (request in 202)

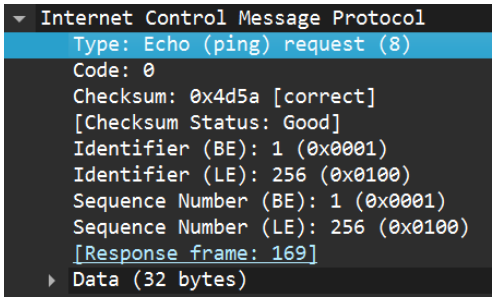
一共 20 个数据包，其中 10 个为源主机发送的 ping 请求，10 个为源主机收到的 ping 响应。

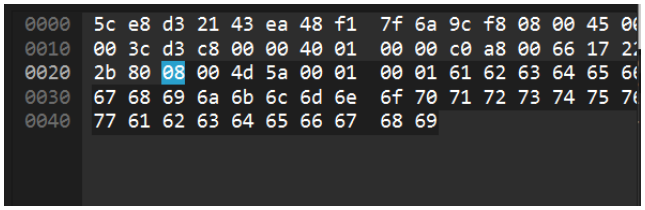
在数据包的内容区域，IP 数据报协议号为 01，即代表使用 ICMP 协议，这意味着 IP 数据报的有效载荷是一个 ICMP 数据包。





该 ICMP 数据包为类型 8、代码 0，即所谓的 ICMP“回显请求”数据包，同时还包含了校验和、标识符及序列号。



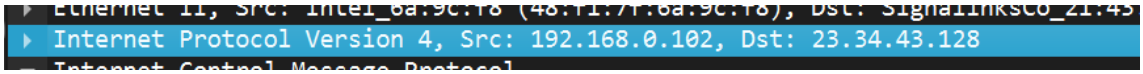


1.3 根据实验结果回答问题

Q1：你的本机 IP 和目标主机的 IP 分别是什么

本机 IP：192.168.0.102

目标主机 IP：23.34.43.128



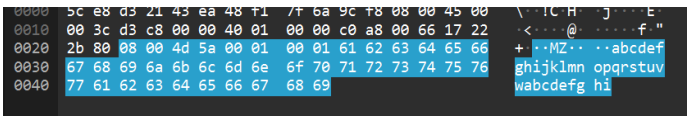
Q2：为什么 ICMP 数据包没有源端口和目的端口号？

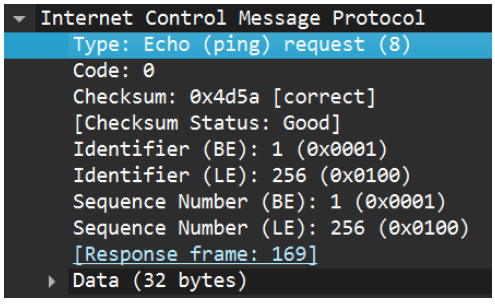
ICMP 是网络层的协议，而端口号是传输层的概念。端口号的作用是 TCP 或 UDP 协议用来区分同一台主机上不同进程的，而 ICMP 设计的初衷不是为了在两个应用程序之间进行端到端的通信，而是为了在主机和路由器之间传递控制信息，不需要由进程的监听端口来接受。

Q3：选中一个 Ping Request 包，分析他的 ICMP Type 和 Code 是多少？还有哪些字段？校验和、序列号、标识符分别占多少字节？

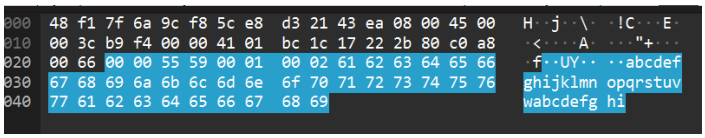
Type 为 8，Code 为 0。还有校验和 0x4d5a、标识符为 1，序列号为 1。

校验和、序列号、标识符各自占 2 字节。

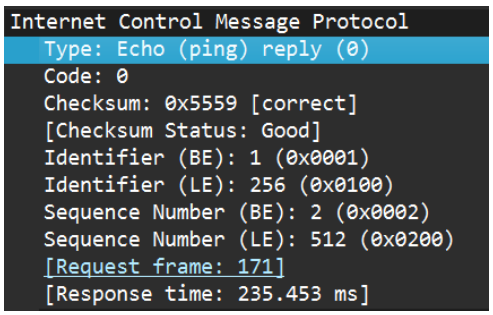




Q4：选中一个 ping reply 包，分析他的 ICMP Type 和 Code 分别是多少，以及上述同样的字段信息。



Type 为 0，Code 为 0，校验和为 0x5559，标识符为 1，序列号为 2



实验任务 2：ICMP AND Traceroute

2.1 了解 Traceroute

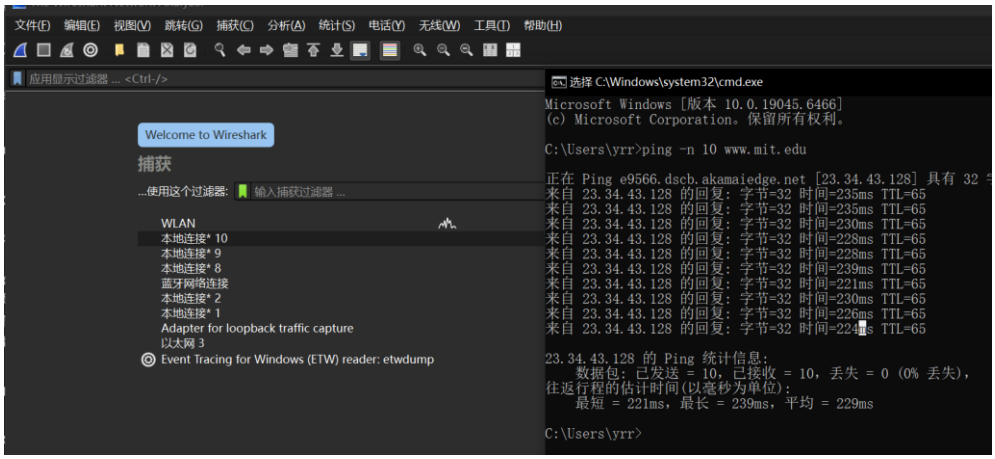
Traceroute 程序可以用来追踪数据包从源到目的地的传输路径。Traceroute 在 Unix/Linux/MacOS 和 Windows 中的实现方式不同。在 Unix/Linux 中，源端会使用一个不太可能的目的端口号向目标发送一系列 UDP 数据包；而在 Windows 中，源端则会向目标发送一系列 ICMP 数据包。

对于这两种操作系统，程序会先发送 TTL =1 的第一个数据包，接着发送 TTL =2 的第二个数据包，依此类推。值得注意的是，路由器会在数据包通过时递减其 TTL 值。当一个数据包到达路由器时他的 TTL =1 ，则路由器会向源端发送一个 ICMP 错误数据包。

在 Windows 操作系统上，使用的是原生的 tracert 程序。一个非常不错的 Windows Traceroute 程序的共享软件版本是 pingplotter（www.pingplotter.com）。我们将在 Wireshark IP 实验中使用 pingplotter，因为它提供了我们所需的一些额外功能。

2.2 使用 cmd tracert 远程主机，并通过 wireshark 获取 ICMP 数据包

2.2.1 启动 cmd 和 wireshark



2.2.2 捕获数据包

启动 wireshark 捕获数据包，再在 cmd 中输入 tracert + homename，此处输入一个跨洋的远程主机，以便于观察路由器跳数、TTL 以及 RTT。跟踪成功后，停止捕获。

经过多次实验，得到的命令行结果如下：

```
C:\Users\yrr>tracert www.inria.fr

通过最多 30 个跃点跟踪
到 inria.fr [128.93.162.83] 的路由:

  1     *      264 ms   260 ms  prod-inriafr-cms.inria.fr [128.93.162.83]

跟踪完成。

C:\Users\yrr>tracert www.inria.fr

通过最多 30 个跃点跟踪
到 inria.fr [128.93.162.83] 的路由:

  1     *        *      261 ms  prod-inriafr-cms.inria.fr [128.93.162.83]

跟踪完成。

C:\Users\yrr>tracert www.inria.fr

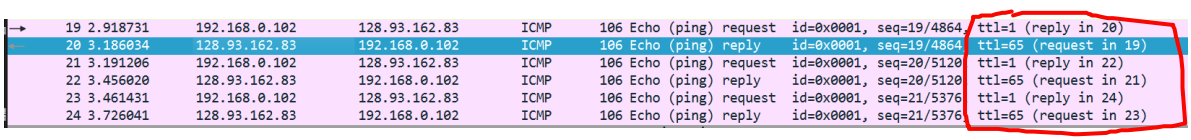
通过最多 30 个跃点跟踪
到 inria.fr [128.93.162.83] 的路由:

  1   267 ms   265 ms   264 ms  prod-inriafr-cms.inria.fr [128.93.162.83]

跟踪完成。
```

发现结果中始终只存在一个路由信息，而我们明确知道目的网址位于法国，这绝对不可能是只经过了一个路由器，在物理上是不可能的现象。从图中我们可以知道延迟时间 RTT 为 264ms，260ms，通常若是局域网（跳数为 1），延迟一般在 1ms 以内，最多不可能超过 10ms，此处的 260ms 通常是跨越了半个地球的光纤传输的物理时延。说明中间的路由“路程”被省略了。

下面为 wireshark 所捕获到的 ICMP 包：



从图中我们可以看到一共捕获了三组 request 和 reply 数据包，request 中 TTL = 1，正常情况下，TTL = 1 的包发给网关，网关将 TTL - 1，从 1 变为 0，然后丢弃该包并返回一个 ICMP 错误报文。当在 reply 中，本机收到的 repaly 源地址为法国的服务器，这意味着本机发出的 TTL = 1 的包，穿过了中间几十个路由器，到达了法国，并且对方服务器还给了回信。

通过查询资料，发现这是典型的 VPN/隧道特征，可能原因如下：

1、未关闭 VPN，导致网络流量被封装进一个“隧道”，直接从本机发送到隧道的出口服务器，对于 tracert 程度来说，看不到中间经过的路由器，只能看到进隧道和出遂道这一跳。

2、关闭了 VPN 的情况下，仍旧如此显示，可能是虚拟网卡未复位，流量走的依然是虚拟网卡，数据包被封装在另一个协议里，直接到了 VPN 出口，在隧道内不，TTL 计数可能没有被正确处理，或者被封装层屏蔽了。

3、路由器级 VPN：本机连接的 WiFi 路由器本身刷了 OpenWrt 并且挂了梯子，该情况下，隧道实在路由器上建立的。

4、系统代理残留：某些代理软件退出不彻底，残留了路由表规则。

2.2.3 再次捕获数据包

本次更换手机热点，来尝试连接 www.mit.edu。由于手机网络通常是“较为纯净”的公网环境，在正常情况下能看到真实的跃点列表。

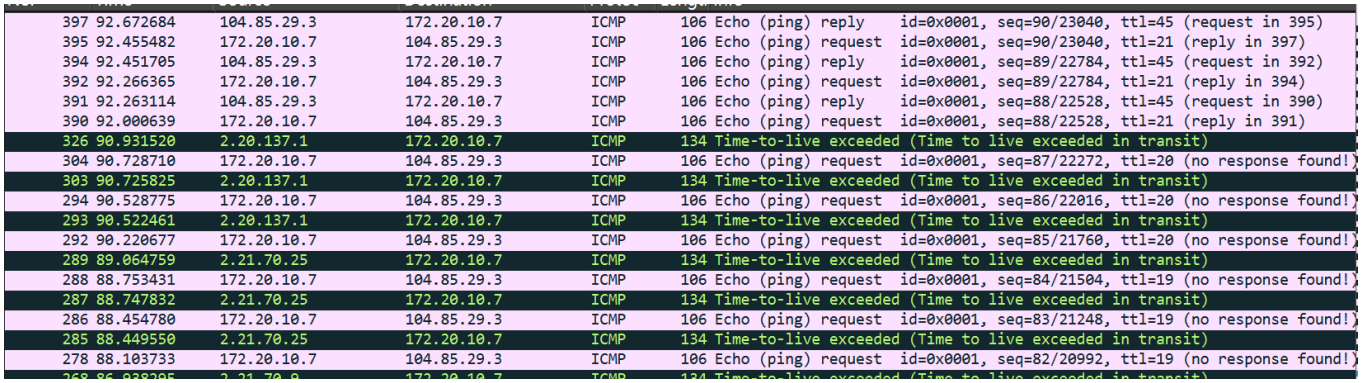
cmd 命令行跟踪和 wireshark 捕获结果如下：

```
C:\Users\yrr>tracert www.mit.edu

通过最多 30 个跃点跟踪
到 e9566.dscb.akamaiedge.net [104.85.29.3] 的路由:

  1     2 ms     2 ms     2 ms  172.20.10.1
  2     *        *        *     请求超时。
  3     *        *        *     请求超时。
  4     *        *        *     请求超时。
  5     *        *        *     请求超时。
  6     *        *       48 ms  119.167.86.77
  7     *        *        *     请求超时。
  8     *        *        *     请求超时。
  9     *        *        *     请求超时。
 10     *        *      325 ms  219.158.5.174
 11     *        *        *     请求超时。
 12   228 ms   202 ms   201 ms  be8232.ccr42.fra05.atlas.cogentco.com [154.54.38.114]
 13   376 ms   302 ms   302 ms  be7946.ccr42.par01.atlas.cogentco.com [154.54.72.117]
 14   283 ms   301 ms   301 ms  be9082.rcr82.par05.atlas.cogentco.com [154.54.62.209]
 15   305 ms   303 ms   304 ms  be9072.nr71.b073979-0.par05.atlas.cogentco.com [154.25.12.182]
 16     *      325 ms   203 ms  149.6.167.5
 17   230 ms   285 ms   298 ms  lo1.r02.stem01.par04.fab.netarch.akamai.com [2.21.70.21]
 18   275 ms   200 ms   297 ms  lo1.r02.spine02.par04.fab.netarch.akamai.com [2.21.70.9]
 19   346 ms   293 ms   311 ms  lo1.r02.leaf01.par04.fab.netarch.akamai.com [2.21.70.25]
 20   302 ms   197 ms   203 ms  vlan100.r14.tor01.par04.fab.netarch.akamai.com [2.20.137.1]
 21   262 ms   185 ms   217 ms  a104-85-29-3.deploy.static.akamaitechnologies.com [104.85.29.3]

跟踪完成。
```



397	92.672684	104.85.29.3	172.20.10.7	ICMP	106 Echo (ping) reply id=0x0001, seq=90/23040, ttl=45 (request in 395)
395	92.455482	172.20.10.7	104.85.29.3	ICMP	106 Echo (ping) request id=0x0001, seq=90/23040, ttl=21 (reply in 397)
394	92.451705	104.85.29.3	172.20.10.7	ICMP	106 Echo (ping) reply id=0x0001, seq=89/22784, ttl=45 (request in 392)
392	92.266365	172.20.10.7	104.85.29.3	ICMP	106 Echo (ping) request id=0x0001, seq=89/22784, ttl=21 (reply in 394)
391	92.263114	104.85.29.3	172.20.10.7	ICMP	106 Echo (ping) reply id=0x0001, seq=88/22528, ttl=45 (request in 390)
390	92.000639	172.20.10.7	104.85.29.3	ICMP	106 Echo (ping) request id=0x0001, seq=88/22528, ttl=21 (reply in 391)
326	90.931520	2.20.137.1	172.20.10.7	ICMP	134 Time-to-live exceeded (Time to live exceeded in transit)
304	90.728710	172.20.10.7	104.85.29.3	ICMP	106 Echo (ping) request id=0x0001, seq=87/22272, ttl=20 (no response found!)
303	90.725825	2.20.137.1	172.20.10.7	ICMP	134 Time-to-live exceeded (Time to live exceeded in transit)
294	90.528775	172.20.10.7	104.85.29.3	ICMP	106 Echo (ping) request id=0x0001, seq=86/22016, ttl=20 (no response found!)
293	90.522461	2.20.137.1	172.20.10.7	ICMP	134 Time-to-live exceeded (Time to live exceeded in transit)
292	90.220677	172.20.10.7	104.85.29.3	ICMP	106 Echo (ping) request id=0x0001, seq=85/21760, ttl=20 (no response found!)
289	89.064759	2.21.70.25	172.20.10.7	ICMP	134 Time-to-live exceeded (Time to live exceeded in transit)
288	88.753431	172.20.10.7	104.85.29.3	ICMP	106 Echo (ping) request id=0x0001, seq=84/21504, ttl=19 (no response found!)
287	88.747832	2.21.70.25	172.20.10.7	ICMP	134 Time-to-live exceeded (Time to live exceeded in transit)
286	88.454780	172.20.10.7	104.85.29.3	ICMP	106 Echo (ping) request id=0x0001, seq=83/21248, ttl=19 (no response found!)
285	88.449550	2.21.70.25	172.20.10.7	ICMP	134 Time-to-live exceeded (Time to live exceeded in transit)
278	88.103733	172.20.10.7	104.85.29.3	ICMP	106 Echo (ping) request id=0x0001, seq=82/20992, ttl=19 (no response found!)

从图中可以看出，对于每个 TTL 值，源程序会发送三个探测包。Traceroute 会显示每个探测包的往返时间，以及返回 ICMP TTL 超限消息的路由器的 IP 地址

2.3 根据实验结果回答问题

Q1：本机的 IP 和目的主机 IP 分别是什么？

本机 IP：172.20.10.7

目的主机 IP：104.85.29.3

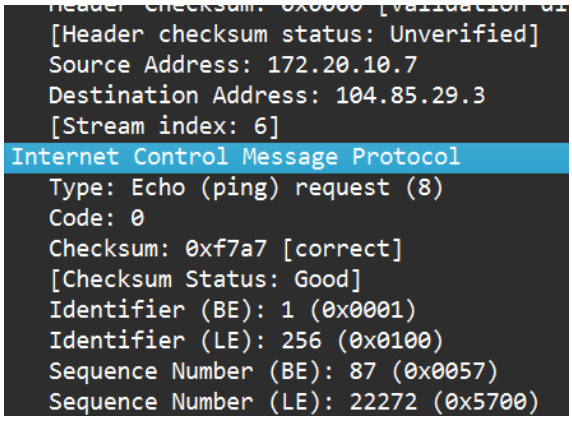
Q2：如果 ICMP 改用 UDP 发送，IP 协议号还会使 01 吗？如果不是，会是多少？

不会是 01，IP 协议字段是用于标识上层协议，如果使用 UDP，协议号会变为 17.

Q3：比较一下 Traceroute 发出的 ICMP Echo 包（探测包）和第一部分 Ping 发出的 Echo 包，有什么不同？

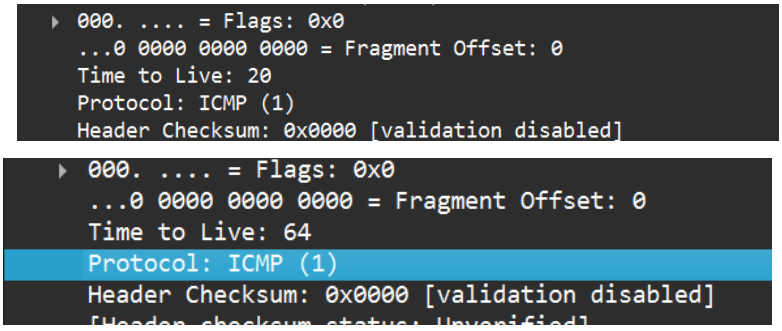
Traceroute 发送的 echo 包依旧还是使用 ICMP，协议号为 1，Type 为 8，Code 为 0，拥有校验和、标识符、序列号字段。

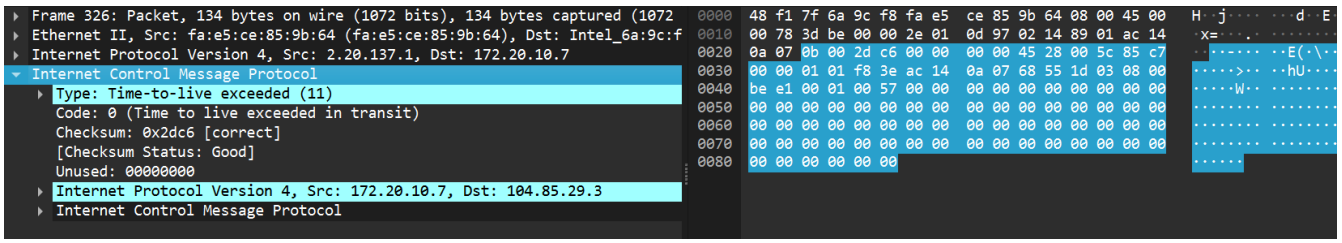
不同点在于 TTL 字段值不一样，ping 实验中，TTL 通常为固定值 64，而 traceroute 中 TTL 是递增的



Q4：观察一个 ICMP Error packet (TTL 超时报文)，它比普通的 Echo 包多了哪些字段？这些字段里包含了什么内容？

多包含了造成错误的那个原始 IP 数据包的头部，一共 20 字节，以及原始 ICMP 数据包的前 8 个字节，加起来一共多包含了 28 字节内容





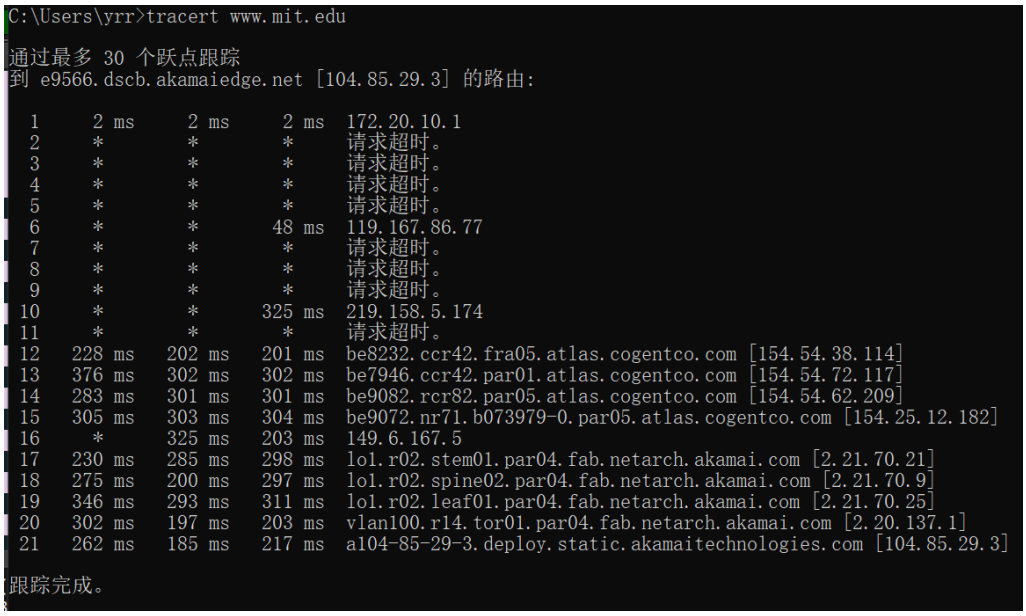
Q5：观察最后收到的 3 个 ICMP 包，它们和之前的 ICMP Error 包有什么不同？为什么不同？（提示：这通常意味着到达了最终目的地）。

最后收到的三个包都是 reply 报文，Type = 0，Code = 0，而之前 ICMP Error 包是 Time-to-live exceeded 报文，因为这三个的 request 包最终到达了目的主机，因此不会像经过路由器时将 TTL 减为 0 而丢弃，目的主机会正常处理这个 Echo Request，并返回一个标准的 Echo reply 包，标志着 Traceroute 跟踪完成。

Q6：在 tracert 的输出结果中，是否存在某一段链路的延迟（Delay）显著长于其他链路？根据路由器的名字，你能猜出这两个路由器位于什么地理位置吗？（例如：跨越了大西洋）。

根据输出结果，可以看到在第 6 跳时，延迟只有 48ms，而到了第 10 跳，延迟突然到了 325ms，之后基本稳定在 250 附近，中间近 200ms 的延迟增量，意味着数据包跨越了极长的物理距离。

第 12 跳的路由器名称 fra05，通常代表德国的法兰克福；第 13 到 20 跳的路由器名称 par01 通常代表法国巴黎。可以猜测中间是连接亚洲和欧洲的海底光缆。



小结

本次实验利用 Wireshark 抓包工具和 Windows 命令提示符（CMD），深入剖析了 ICMP 协议在网络诊断工具 Ping 和 Tracert 中的具体应用。

- 在 Ping 实验中：我通过向目标主机发送 ICMP Echo Request 报文，成功捕获并分析了对应的 Echo Reply 报文。验证了 ICMP 报文位于 IP 数据报的数据部分，且通过 Identifier（标识符）和 Sequence Number（序列号）字段来匹配请求与响应，而非使用传输层的端口号。
- 在 Tracert 实验中：我利用 Tracert 程序通过递增 IP 首部中的 TTL（生存时间），引发沿途路由器丢弃数据包并返回 ICMP Time-to-live Exceeded 差错报文，从而成功追踪到了从本机到目标主机的完整路径。

通过本次实验的学习，从实践的角度，再次重温、复习了理论课上有关于 ICMP 和 IP 报文结构的知识，明确了 ICMP 是网络层协议（IP 协议号为 1），不依赖于传输层的 TCP 或 UDP，因此没有源端口和目的端口；掌握了通过 Type 和 Code 字段区分不同的 ICMP 消息类型（如 Type 8 为请求，Type 0 为应答，Type 11 为 TTL 超时）。详细了解了 Ping 和 Traceroute 的操作原理和操作过

程，理解了 Traceroute 巧妙利用“错误”来探测路径的设计思想：故意发送 TTL 不足的包，诱导路由器报错，从而获取路由器 IP。